EM-Merge: Consolidating Fragmented Templates in LLM Log Parsing

Qiyue Zhu[1 [0009−0004−8815−5716]], Jinyuan Wang[1], Yani Yang[1 [0009−0006−1342−3020]], and Tong Li[1⋆ [0000−0002−8881−0037]]

Beijing University of Technology, Beijing, China
qiyue.zhu@ucdconnect.ie, wangjinyuan@emails.bjut.edu.cn,
yani.yang@ucdconnect.ie, litong@bjut.edu.cn

Abstract. Log parsing is a critical step for log analysis, responsible for converting raw, semi-structured log messages into structured templates. Despite using semantic knowledge to enhance template extraction, LLM-based log parsers frequently generate fragmented templates, generating several marginally distinct templates for semantically identical events. This fragmentation generates duplicate downstream processing, reduces cache hit rates, and inflates the template pool. We introduce EM-Merge, a lightweight, model-independent postprocessing layer that directly mitigates fragmentation without requiring further LLM calls. EM-Merge employs a two-stage strategy: (1) semantic screening, where newly generated and cached templates are encoded with Sentence-BERT and nearest-neighbor search selects semantically plausible candidates; (2) confidence-weighted structural merging, where candidate pairs are scored by a normalized longest-common-subsequence measure modulated by LLM generation confidences, then clustered and merged into generalized templates (non-common tokens replaced by wildcards). This design preserves semantic distinctions while preventing misleading splits driven by punctuation or surface variation. We evaluate EM-Merge on 14 public log datasets and find that, with no runtime overhead, it reliably reduces template redundancy, improves grouping and parsing accuracy, and lowers end-to-end processing cost when compared to strong baselines.

Keywords: Log Parsing · Large Language Models · Postprocessing

1 Introduction

Large-scale software systems, such as cloud services, distributed storage clusters, and internet-scale applications, produce massive volumes of semi-structured log data that record every system event, performance statistic, and error condition. Critical processes including capacity planning, failure prediction, anomaly detection, and root-cause analysis are supported by these logs [1,2,3]. However, raw log messages are a combination of fixed text and variable fields, making direct analysis both inefficient and error-prone. Log parsing addresses this by extracting a template (the invariant scaffold of a message) and its parameters (the dynamic elements). For example, `"User Alice logged in from 192.168.0.1"` is

⋆ Corresponding author. ✉ litong@bjut.edu.cn

parsed into the template `"User <*> logged in from <*>"`, with `"Alice"` and `"192.168.0.1"` identified as parameters. Such structured output is essential for reliable downstream processing by ML and rule-based systems.

Recent advances have applied large language models (LLMs) such as GPT-3 [4] and its offspring to log parsing via prompt-based, zero-, and few-shot learning [5]. These methods excel at handling novel formats and capturing subtle semantics. Yet, two core challenges remain unaddressed in existing LLM parsers:

1. Template Fragmentation. A single event type, meaning log messages that describe the same underlying system action, often produces multiple slightly different templates due to minor token order shifts, synonym choices, or wildcard insertions. This "fragmentation" bloats the template set, reduces cache reuse, and complicates clustering for anomaly grouping.
2. Lack of Lightweight Consolidation. While some approaches attempt offline merge or split operations [6], they typically require repeated calls to the LLM or manual intervention, incurring high latency and resource costs that make real-time deployment impractical.

To overcome these limitations, we introduce EM-Merge, a lightweight, model-independent postprocessing framework that can be retrofitted onto any cache-based LLM parser without additional LLM inference. Our design rationale is as follows: (1) Semantic Screening. Recognizing that LLMs can misinterpret punctuation and domain cues, we first embed each newly generated and cached template into a dense vector space using SentenceBERT [7]. By performing a fast nearest-neighbor search, we select only the most semantically similar template candidates, dramatically reducing downstream computational cost. (2) Confidence-Weighted Structural Merging. Within this candidate set, we compute a merge score that blends normalized Longest Common Subsequence (LCS) overlap with the LLM's own confidence estimates. Templates that exceed a tunable threshold are then grouped via density-based clustering and merged into a single generalized template by preserving common tokens and masking divergent ones. By explicitly combining semantic embeddings with structural and confidence signals, EM-Merge unifies fragmented templates into coherent representations, improving cache hit rates and simplifying downstream analysis, all with minimal performance overhead.

We validate EM-Merge on fourteen public datasets from the LogPAI Loghub-2.0 collection [3,8], demonstrating substantial reductions in template redundancy and notable gains in end-to-end accuracy and throughput compared to state-of-the-art parsers.

To summarize, this paper makes the following contributions:

- We propose EM-Merge, a novel postprocessing pipeline that consolidates semantically equivalent log templates without requiring additional calls to large language models.
- We develop a semantic screening mechanism based on Sentence-BERT to efficiently preselect candidates for merging.

– We introduce a confidence-weighted structural similarity metric combined with a clustering strategy to achieve precise fusion of log templates.
– We perform comprehensive evaluations on fourteen real-world datasets, demonstrating consistent improvements in template coherence and parsing efficiency.
– The source code of EM-Merge is publicly available at https://github.com/VictoriaZQY/EM-Merge to benefit both practitioners and researchers in the field of log analysis.

2 Background

In this section, we review the two foundational pillars of our work. First, we present the evolution of log parsing techniques in software systems, then demonstrate the emergence and capabilities of large language models (LLMs) in natural language processing. Together, these perspectives motivate the design of EM-Merge, a lightweight, embedding-aware postprocessing framework for log template consolidation.

2.1 Log Parsing

Modern software systems continuously generate large volumes of semi-structured logs that record runtime behaviors such as system events, errors, and user interactions [9,10]. To support downstream tasks like anomaly detection or root cause analysis, these logs must first be converted into structured formats. This process is known as log parsing, which separates constant patterns (templates) from variable content (parameters) [11,12].

```
// Logging statements
logger.info(String.format("%s : Scanning mode %s", time, mode));
logger.info(String.format("button report: %s", reportCode));
logger.warn(String.format("Couldn't find a requirement string for
masquerading client %s", clientName));
```

E607 201 10:13:34.922 : Scanning mode Contacts Only
E606 633 button report: 0x80039B7
E312 19 Couldn't find a requirement string for masquerading client com.apple.Maps

EventId	Occurrences	Content	EventTemplate
E607	201	10:13:34.922 : Scanning mode Contacts Only	<*> : Scanning mode <*>
E606	633	button report: 0x80039B7	button report: <*>
E312	19	Couldn't find a requirement string for masquerading client com.apple.Maps	Couldn't find a requirement string for masquerading client <*>

Fig. 1: An example of log parsing

For example, in Figure 1, the message: "`10:13:34.922 : Scanning mode Contacts Only`" can be parsed into the template "`<*> : Scanning mode <*>`" where "`10:13:34.922`" and "`Contacts Only`" become dynamic fields. Once structured, these templates enable downstream machine learning and rule-based analytics to be applied more reliably.

Traditional log parsers often rely on manually crafted regular expressions, which lack scalability and generalizability across evolving systems [13]. To address this, a variety of automated, data-driven methods have emerged:

- Frequent pattern mining methods (e.g., Logram [14]) assume that static tokens occur more frequently and extract templates accordingly.
- Clustering-based approaches (e.g., LenMa [15]) group similar log lines using token-based distances but often require sensitive threshold tuning.
- Heuristic parsers (e.g., Drain [9]) build parse trees based on token positions and delimiters, offering a balance between efficiency and flexibility.

More recently, semantic-based methods have introduced neural models to capture contextual meaning within logs, enabling improved discrimination between static and dynamic parts. For example, sentence-level embeddings (e.g., SentenceBERT variants) have been used to represent log events for downstream tasks such as anomaly detection and semantic grouping, demonstrating that semantic representations help identify paraphrased or semantically equivalent events that token-only methods miss [7,16]. These methods typically rely on labeled data for supervised learning and show promise in handling complex log structures, though their practical deployment is often limited by annotation costs and sensitivity to domain shift [17,18].

2.2 Large Language Models for Log Parsing

Transformer-based LLMs have transformed many NLP tasks by providing strong contextual representations and by enabling few-shot or in-context learning: large pretrained models can often perform new tasks with only a handful of examples or simple prompts, reducing dependence on large hand-labeled corpora [4]. These capabilities translate naturally to log parsing: LLMs and LLM-derived encoders capture semantic relations that allow them to recognize when differently worded log messages share the same underlying template, and they can be prompted or few-shot tuned to locate variable fields with much less annotation than conventional supervised methods. Recent works demonstrate several concrete benefits from this shift. First, LLM-based parsers produce rich, reusable artifacts, such as line- or token-level embeddings, token confidences, and alternative candidate parses that enable powerful downstream operations (e.g., embedding-based clustering, confidence-weighted selection, and caching) beyond a single template output. Second, LLM-centric systems (e.g., LogPPT [19], LILAC [17] and subsequent LLM-parsing studies) combine prompt or few-shot strategies with caching or selective invocation to achieve high parsing accuracy while mitigating labeling cost and improving robustness across datasets [20].

At the same time, practical deployment of LLM-based parsing faces clear constraints: repeated LLM calls over high-volume streams incur latency and monetary cost, and models can produce inconsistent templates across repeated or slightly different prompts, which harms cache hit rates and downstream stability. These trade-offs have motivated hybrid designs that treat the LLM as a semantic signal generator rather than an end-to-end oracle, i.e., systems extract embeddings and confidences from a limited set of LLM inferences and then apply lightweight, efficient postprocessing (caching, embedding pre-screening, and confidence-weighted merging) to consolidate templates and avoid further expensive model calls [17,19,21]. EM-Merge follows this hybrid philosophy: by reusing embeddings and token-confidence information emitted by LLM-based parsers and by applying an efficient, confidence-aware merging strategy, EM-Merge aims to improve cross-call consistency and cache effectiveness without invoking additional LLM inference for consolidation.

3 Approach

Based on our preliminary evaluation using ChatGPT (gpt-3.5-turbo-0613) in zero-shot parsing over the Loghub-2.0 datasets, we found that LLMs often misinterpret domain cues (e.g., splitting hexadecimal fields after colons), fail to generalize recurring structural patterns, and produce fragmented templates for semantically identical events. These issues inflate the template inventory and reduce grouping accuracy. ChatGPT's excessive sensitivity to punctuation signals and irregular structural anchoring, which were also observed in earlier LLM-based parsing studies [22], were responsible for almost all of these errors.

As illustrated in Figure 1, the raw message `10:13:34.922 : Scanning mode Contacts Only` is correctly abstracted into `<*> : Scanning mode <*>`, where the timestamp and mode vary while the rest remains stable. This parameterization is acceptable because it preserves the invariant event structure. However, excessive sensitivity to punctuation can lead to incorrect variants such as `10:13:34.<*> : Scanning mode Contacts Only` or `<*> : Scanning mode Contacts Only`, which either break a token boundary or remove a useful anchor. Such cases fragment the event pattern and lower parsing consistency.

A model that understands sentence structure and semantics could easily avoid such issues and produce the correct segmentation shown in Figure 1. This observation motivates the design of EM-Merge, which was designed to (a) use SentenceBERT embeddings to capture semantic similarity beyond token-level signals [7], (b) apply a confidence-modulated structural similarity metric to merge only templates that align both semantically and structurally, and (c) cluster and merge similar templates. Together, these stages form a lightweight postprocessing layer that repairs fragmented templates before they are cached, improving consistency and reducing redundant LLM invocations.

3.1 Overview and Motivation

Figure 2 presents the overall design of the EM-Merge workflow.

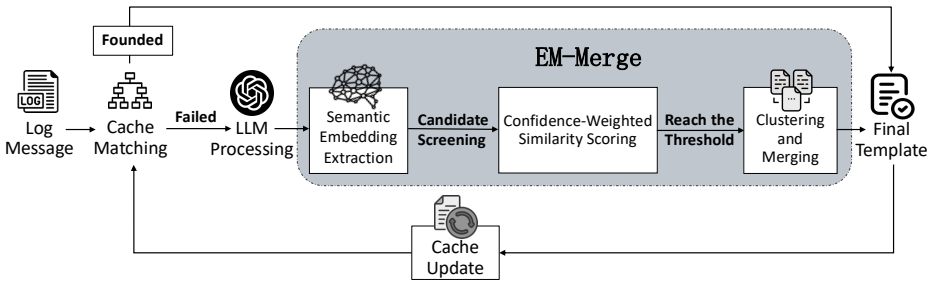


Fig. 2: Overview of the Workflow

When a log message arrives, it first enters the parser and is checked against the parsing cache. If a confident cache hit is found, the cached template is returned immediately, and the system moves on to the next message. If no adequate match exists, the system retrieves a small set of relevant cached templates and uses them as candidate demonstrations for the in-context learning enhanced parser, which queries the LLM and obtains a proposed template together with any available token-level confidence signals. However, this raw LLM output is not always correct. For example, in Figure 3, the model may produce a fragmented form such as `"10:13:34.<*> : Starting AirDrop server for user <*> on wake"` or even an over-abstracted `"<*> : Starting AirDrop server for user <*>"`, both of which mishandle punctuation and token boundaries. Instead of committing that raw output to the cache, EM-Merge takes over and encodes the generated template and the retrieved candidates into dense semantic vectors so that meaning, not just surface tokens, guides subsequent decisions. These embeddings are used to pre-screen plausible merge candidates, then a confidence-weighted structural similarity test compares token alignments with greater emphasis on high-confidence elements and downweights uncertain or model-generated parts. Candidates that pass this combined semantic and structural scrutiny are grouped by clustering, and clusters that meet consolidation thresholds are merged into a single canonical template. The parsing cache is then updated transactionally so that fragmented duplicates are replaced by the consolidated template, and future messages benefit from higher cache hit rates. After EM-Merge's screening, scoring, clustering, and merging steps, the erroneous or fragmented candidate is repaired to the correct canonical form shown in Figure 3 (for example, `"<*> : Starting AirDrop server for user <*> on wake"`), so subsequent cache matches return a consistent template.

This whole sequence, including cache match, selective LLM invocation, embedding screening, confidence-aware comparison, clustering, and cache update, repeats for every incoming message, forming a continuous processing loop. Each iteration both produces parsed output and yields signals such as new templates, confidence scores, and merge decisions that adapt the cache over time. When enough new evidence accumulates, the loop will drive more substantial consolidation decisions so the cache remains compact and consistent as log distributions evolve, and then the system returns to handling the next incoming log.

Specifically, EM-Merge contains 5 steps:

- **Semantic Embedding Extraction** overcomes LLMs' punctuation sensitivity by encoding template semantics into dense vector representations, capturing contextual meaning beyond surface tokens.
- **Candidate Screening** eliminates exhaustive $O(n^2)$ comparisons through approximate nearest-neighbor search, reducing computational overhead by 2-3 orders of magnitude while preserving recall.
- **Confidence-Weighted Similarity Scoring** fuses structural alignment with the LLM's intrinsic confidence signals, creating a noise-resistant similarity metric that prioritizes high-quality merges.
- **Clustering and Merging** discovers natural template groupings using DBSCAN, accommodating arbitrary cluster shapes while filtering outlier templates.
- **Cache Update** ensure transactional replacement of fragmented templates with consolidated versions, maintaining parsing consistency.

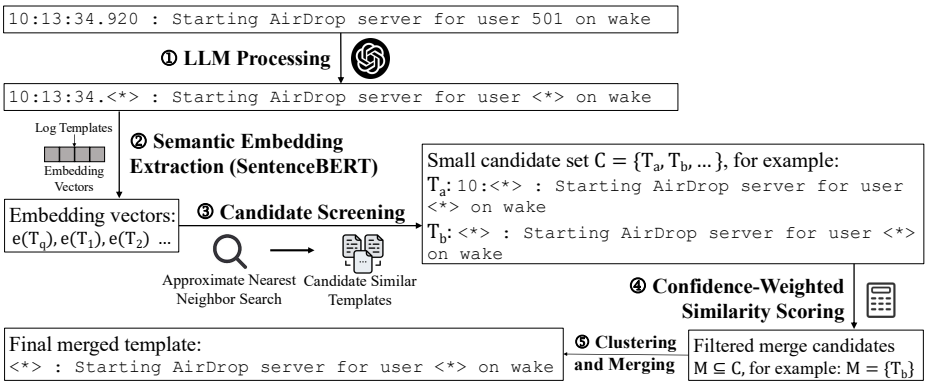


Fig. 3: Overview of EM-Merge

Figure 3 shows the EM-Merge pipeline for a single parsed result. When the LLM returns a new candidate template T_q, the system first converts every template into a fixed-length numeric representation commonly called a dense vector. By "dense vector", we mean an embedding produced by a pretrained sentence encoder like SentenceBERT, a compact array of real numbers (often hundreds of dimensions) that places semantically similar templates near one another in Euclidean or cosine space. These embeddings make it efficient to retrieve a small set of semantically related cache entries via an approximate nearest-neighbor search (for example, using an ANN index such as Faiss). The retrieved set C is the pool of candidates that will be examined more carefully.

For each candidate T_i in the retrieved set C, we evaluate two complementary signals: semantic proximity (which is already implied by the neighbor search) and structural agreement. Structural agreement is measured by a normalized longest common subsequence (LCS) computed over tokenized templates. Concretely, templates are tokenized with a consistent rule so that punctuation, timestamps, and placeholders are handled predictably. We then compute the LCS length

between the token sequences of T_q and T_i. To make this comparable across templates of different lengths, we divide the LCS length by the length of the longer template, yielding a score between zero and one that captures how much ordered token sequence material the two templates share. For example, given the raw message ‘‘`10:13:34.920 : Starting AirDrop server for user 501 on wake`’’, the LLM might produce slightly different templates that nonetheless share the backbone ‘‘`: Starting AirDrop server for user ... on wake`’’, and the normalized LCS highlights that shared backbone even when numeric or timestamp tokens differ.

EM-Merge combines this structural measure with the LLM's confidence signals to form a confidence-weighted similarity score. We first reduce token-level confidences to a template-level confidence (for instance, by averaging per-token confidences after ignoring wildcard placeholders). Pairs where the model expresses very different certainty should be merged more cautiously, since high agreement in confidence increases merge propensity, and large differences decrease it. For a candidate T_i, we compute the absolute difference between its template confidence and the confidence of T_q, and a large difference indicates disagreement in certainty and should reduce the propensity to merge. In practice, we combine the two signals with a simple convex mixture. The score increases with normalized LCS and with close agreement in confidences, and candidates whose combined score exceeds a chosen threshold form the merge set M.

The merge set is then clustered to find groups of mutually compatible templates using a density-based clustering such as DBSCAN on the filtered candidates. Within each cluster, we create a consolidated template by keeping the longest common subsequence as the stable backbone and replacing tokens that differ across cluster members with a wildcard token (for readability, we show the wildcard as `<*>`) so that variable fields are abstracted while shared structure is preserved. These consolidated templates replace the originals in the cache in a transactional update, ensuring that the cache remains consistent and that subsequent logs benefit from higher hit rates.

3.2 Semantic Embedding Extraction

Our preliminary analysis revealed that LLM-generated templates often misinterpret punctuation and word semantics (e.g., colons and parentheses) and produce fragmentary templates as a result. In contrast, humans leverage semantic awareness to identify variables as single units. To overcome this, we incorporate a semantic embedding stage to capture the global meaning of each template. We chose Sentence-BERT [7] for this task because it transforms templates into fixed-length vector representations using a Siamese/Triplet architecture built upon BERT. Sentence-BERT is an enhanced version of the pre-trained BERT model that uses Siamese and triplet network architectures to produce semantically meaningful sentence embeddings. This approach greatly improves the efficiency of semantic similarity computations, reducing the computational cost from over 65 hours with naive BERT to just a few seconds for thousands of templates while preserving semantic fidelity [7]. SentenceBERT complementarily fills

the gap left by ChatGPT's sequence generation, as the former encodes semantic content holistically, whereas the latter focuses on token-by-token generation [23]. Formally, each template T is mapped to an embedding vector:

$$\mathbf{e}(T) = \text{SentenceBERT}(T) \in \mathbb{R}^d,$$

These embeddings serve as the foundation for downstream similarity evaluation.

3.3 Embedding-Based Candidate Screening

Although SentenceBERT embeddings capture semantics, comparing every new template T_q against thousands of cached ones would be prohibitively expensive. We therefore perform semantic filtering via an approximate nearest-neighbor (ANN) index to retrieve only those candidates likely to merge, thus focusing subsequent expensive operations on a small set. All cached embeddings $\{\mathbf{e}(T_i)\}$ are indexed in Faiss [24]. This allows us to retrieve only those templates whose embeddings lie within a Euclidean distance threshold ϵ:

$$\|\mathbf{e}(T_q) - \mathbf{e}(T_i)\|_2 < \epsilon.$$

By tuning ϵ, we reduce the candidate set from $O(n)$ to a manageable size $k \ll n$, balancing recall and efficiency. In practice, we selected $\epsilon = 0.3$ as the operating point by grid search on a development split and validated it on the HPC benchmark. As shown in Section 5.3 RQ2: Efficiency (Table 3), $\epsilon = 0.3$ gave the best practical trade-off in our experiments since it produced the lowest end-to-end parse time while preserving parsing and grouping accuracy, and thus was used across the reported evaluations.

3.4 Confidence-Weighted Structural Similarity

Semantic closeness alone may admit false positives (e.g., semantically related but structurally distinct templates), while structural overlap alone may ignore high-quality semantic matches. To refine, we blend structure with the parser's own confidence signals. For each candidate pair $(T_q, T_i) \in \{T_q\} \times \mathcal{C}$, we compute:

- $\text{LCS}(T_q, T_i)$: length of the longest common subsequence over tokenized templates.
- $|T_q|$, $|T_i|$: token counts of the two templates.
- $c_q, c_i \in [0, 1]$: average per-token confidence scores returned by the original LLM when generating each template.

To ensure high-quality merging, we define the *confidence-weighted similarity*:

$$\text{sim}_{\text{cw}}(T_q, T_i) = \frac{\text{LCS}(T_q, T_i)}{\max(|T_q|, |T_i|)} \cdot \Big(1 - \alpha\,|c_q - c_i|\Big),$$

where $\alpha \in [0, 1]$ modulates the influence of confidence differences. Note that the parameter α encodes our belief in the LLM's confidence scores. An α=0.0

degenerates the similarity to pure structural overlap, while a large α (e.g., >0.5) amplifies confidence effects. Through ablation studies, we found that α=0.3 best preserves high-quality matches while suppressing spurious low-confidence ones. Pairs with $\text{sim}_{\text{cw}} \geq s_{\text{cw}}$ are promoted to the merge candidate set $\mathcal{M}$. This threshold determines merging strictness: A high threshold (e.g., 0.9) ensures near-identity before merging, while a lower threshold (e.g., 0.7) admits more variation. We select $s_{\text{cw}} = 0.8$ based on maximizing cluster cohesion (Silhouette Score) and parsing accuracy improvements across multiple datasets.

Longest Common Subsequence (LCS) is a well-established measure for identifying structural overlap, and by weighting with confidence differences, we mitigate errors where one template may be under-confident. A higher confidence agreement boosts similarity; if $\text{sim}_{\text{cw}} \geq s_{\text{cw}}$ (e.g., 0.8), the pair enters merge consideration.

The choice of ϵ determines the balance between recall and efficiency. In our experiments, values between 0.6 and 0.8 yielded the best balance: smaller ϵ led to high precision but missed potential matches; larger ϵ captured more matches but increased false positives. We default to ϵ=0.75 based on development set performance.

3.5 Clustering and Merging

Multiple candidate pairs may collectively describe a single event type. A clustering step groups interconnected templates into merge clusters, ensuring each event category yields exactly one consolidated template. Instead of pairwise merging, we apply DBSCAN [25], a density-based clustering algorithm well-suited for discovering arbitrarily shaped clusters and ignoring noise, to group semantically and structurally similar templates. Using sim_{cw} as the similarity metric, each cluster $\mathcal{G}$ must satisfy $\text{sim}_{\text{cw}} \geq s_{\text{cw}}$ for at least a minimal number of points (minPts=2). For each cluster, we compute a multi-sequence LCS across all member templates to extract the shared structural skeleton:

$$L = \text{LCS}\big(T_j \mid T_j \in \mathcal{G}\big).$$

Non-overlapping tokens are replaced with the wildcard symbol `<*>` to produce a generalized merged template T_{merged}. This step directly addresses template fragmentation by unifying semantic variants into a single representation. For example, fragmented templates from OpenStack logs such as:

```
"Connected to <*> port <*>"
"Connection to <*> port <*> established"
"Connected <*>:<*>"
```

will yield a shared skeleton L corresponding to the backbone phrase `"Connected to <*> port <*>"`, and the consolidated template becomes

```
"Connected to <*> port <*>"
```

The merged template is therefore the canonical form that best captures the invariant structure present across the cluster members.

3.6 Cache Update

After a cluster produces a consolidated template, the system must make the merged representation the operative template in the parsing cache. To maintain historical continuity, for each clustered set $\mathcal{G}$ we perform an atomic replacement transaction in three steps. First, we create the merged template T_{merged} as described above. Second, we delete the constituent templates $T_j \in \mathcal{G}$ from the active cache and insert T_{merged} in their place. Third, we reassign all historical and in-flight log records that had previously been mapped to any T_j so that they now point to T_{merged}. These reassigned mappings include any metadata used by downstream consumers, such as counts, timestamps, and provenance pointers, which we aggregate or carry over to preserve historical continuity.

Using the OpenStack example above, the cache update replaces the three fragmented entries with the single canonical entry `"Connected to <*> port <*>"` and remaps prior matches of `"Connected <*>:<*>"`, `"Connection to <*> port <*> established"` and similar variants to the new canonical template. This ensures that subsequent cache matching returns the unified representation and that analytics consuming template identifiers observe a consistent template id rather than multiple near-duplicates.

To support correctness and traceability, the replacement is atomic: consumers never observe an intermediate state in which some records point to old templates while others point to the merged template. We also maintain optional backward pointers (aliases) from removed template identifiers to the new canonical identifier for a configurable retention period. These aliases facilitate debugging and allow consumers that have cached template IDs to transition gracefully. If no candidate exceeds the similarity threshold, the new template T_q is simply inserted into the cache as is. Together, these measures keep the template database compact, semantically coherent, and operationally efficient without further LLM interaction.

4 Experimental Design

4.1 Research Questions

We evaluate our approach by answering the following research questions (RQs):

- **RQ1: How effective is EM-Merge compared to existing log parsers?**
- **RQ2: How efficient is EM-Merge in processing large-scale log data?**
- **RQ3: How much do different modules contribute?**

4.2 Datasets

All experiments utilize the large-scale log parsing dataset Loghub-2.0 benchmark [8] from [3], comprising 14 real-world datasets totaling approximately 50

million logs with 3,489 manually verified templates. This collection spans distributed systems (HDFS, Spark), OS kernels (Linux, Windows), server applications (Apache, Zookeeper), and embedded systems (HealthApp), presenting various challenges. Standard preprocessing includes timestamp removal, case normalization, and special character handling following the LogPAI [3,8] pipeline.

4.3 Baselines

To demonstrate our work, we have chosen four open-source, cutting-edge log parsers for comparison with our approach. More specifically, considering their exceptional efficacy relative to all syntax-based log parsers, we select AEL and Drain; regarding the superior parsing accuracy among recent semantic-based log parsers, we choose UniParser [12], LogPPT [19], and LILAC [17] as baselines to implement dynamic demonstration through semantic similarity search. All baselines use official implementations with default parameters, with stochastic methods executing seven runs for statistical reliability.

4.4 Evaluation Metrics

We use four standard metrics from log parsing literature [12,17,26] with refined definitions:

Grouping Accuracy (GA): Measures if each log message's template matches exactly the same log group as ground-truth.

Parsing Accuracy (PA): Assesses if templates and parameters are correctly extracted, requiring exact static token match and perfect variable identification.

F1 of Grouping (FGA): Addresses GA's cluster-size bias by computing the harmonic mean of the Precision of Grouping Accuracy (PGA) and the Recall of Grouping Accuracy (RGA) at the template level.

F1 of Template Accuracy (FTA): Emphasizes correctly recognized templates, defined as those whose tokens fully match the ground-truth and whose assigned logs share the same template. FTA can be calculated as the template accuracy's precision and recall harmonic mean.

4.5 Implementation and Environment

Experiments were performed on a Lenovo Legion Y7000 laptop with an 8-core Intel i7 CPU (2.6 GHz max turbo), NVIDIA RTX 30-series GPU (8GB VRAM), and 24GB DDR4 RAM. The environment utilized Python 3.10 with NumPy/Pandas for core processing, CUDA 11.8 and cuDNN 8.6 for GPU acceleration in baseline models. Experimental constraints included API dependency limiting reproducibility and GPU memory restricting batch sizes. For the large language model component, we used the OpenAI API v1.3 with gpt-3.5-turbo-0613, which provides a 4K token context window and temperature set to 0. We selected GPT-3.5 to align with prior baseline settings and ensure comparability of results, while future work will examine the applicability of more recent models.

5 Experimental Results and Analysis

5.1 RQ1: Effectiveness

Tables 1 and 2 report Grouping Accuracy (GA), Parsing Accuracy (PA), F1 of Grouping (FGA) and F1 of Template Accuracy (FTA) for six parsers over fourteen benchmarks. EM-Merge consistently outperforms both traditional methods (Drain, AEL) and recent LLM-based parsers (UniParser, LogPPT, LILAC). On average, EM-Merge achieves GA=0.92, PA=0.82, FGA=0.93, and FTA=0.76, representing relative improvements over the strongest baseline (LILAC). Notably, on challenging datasets such as OpenSSH and HealthApp, EM-Merge raises FTA by over 10 points, indicating its superior ability to unify fragmented templates without overgeneralizing.

Table 1: Metric comparison with existing log parsers (Part 1)

	Drain				**AEL**				**UniParser**			
	GA	PA	FGA	FTA	GA	PA	FGA	FTA	GA	PA	FGA	FTA
Proxifier	0.69	0.69	0.21	0.18	0.97	0.68	0.67	0.42	0.51	0.63	0.39	0.52
Apache	**1.00**	0.73	**1.00**	0.52	**1.00**	0.73	**1.00**	0.52	0.99	0.96	0.84	0.25
OpenSSH	0.71	0.59	0.87	0.49	0.70	0.36	0.69	0.33	0.22	0.55	0.06	0.04
HDFS	**1.00**	0.62	0.94	0.61	**1.00**	0.62	0.76	0.56	0.52	0.95	0.01	0.01
OpenStack	0.75	0.03	0.01	0.00	0.74	0.03	0.68	0.17	0.99	0.52	0.86	0.26
HPC	0.79	0.72	0.31	0.15	0.75	0.74	0.20	0.14	0.78	0.95	0.60	0.33
Zookeeper	0.99	0.84	0.90	0.61	**1.00**	0.84	0.79	0.47	**1.00**	**0.99**	0.67	0.48
HealthApp	0.86	0.31	0.01	0.00	0.73	0.31	0.01	0.00	0.46	0.79	0.72	0.35
Hadoop	0.92	0.54	0.79	0.38	0.82	0.54	0.12	0.06	0.75	**0.85**	0.79	0.55
Spark	0.89	0.39	0.86	0.41	–	–	–	–	0.83	0.72	0.00	0.00
BGL	**0.92**	0.41	0.62	0.19	**0.92**	0.41	0.59	0.17	0.91	**0.95**	0.61	0.21
Linux	0.69	0.11	0.78	0.26	**0.92**	0.08	0.81	0.22	0.29	0.16	0.44	0.21
Mac	0.76	0.36	0.23	0.07	0.80	0.25	0.79	0.21	0.76	0.58	0.69	0.27
Thunderbird	**0.83**	0.22	0.24	0.07	0.79	0.16	0.12	0.04	0.49	0.41	0.10	0.05
Average	0.84	0.47	0.55	0.28	0.86	0.44	0.56	0.25	0.68	0.72	0.48	0.25

5.2 RQ2: Efficiency

To assess the runtime efficiency of EM-Merge in a realistic setting, we measured end-to-end parsing time on all 14 Loghub-2.0 datasets using our reference platform. Considering the average total parsing time (in seconds) for the base LILAC parser versus LILAC augmented with EM-Merge, LILAC alone required on average 1,686.50 s to parse all messages, whereas EM-Merge reduced this to 1,313.80 s. This is a 22.1% decrease in overall processing time.

The runtime improvement stems largely from the embedding-based candidate screening. By tuning the semantic distance threshold ϵ used in the ANN retrieval, we can sharply reduce the number of expensive structural comparisons per template. Table 3 shows a focused sensitivity study on the HPC dataset

Table 2: Metric comparison with existing log parsers (Part 2)

	LogPPT				**LILAC**				**EM-Merge**			
	GA	PA	FGA	FTA	GA	PA	FGA	FTA	GA	PA	FGA	FTA
Proxifier	0.99	**1.00**	0.87	**0.96**	0.52	0.52	0.86	0.86	**1.00**	0.99	**1.00**	0.82
Apache	0.87	0.95	0.56	0.35	**1.00**	**1.00**	**1.00**	**0.86**	**1.00**	**1.00**	**1.00**	**0.86**
OpenSSH	0.28	0.65	0.07	0.09	**0.75**	**0.97**	0.80	0.71	**0.75**	**0.97**	**0.88**	**0.85**
HDFS	0.41	0.90	0.52	0.31	**1.00**	0.95	**0.97**	**0.93**	**1.00**	**1.00**	**0.97**	0.90
OpenStack	0.56	0.41	0.89	0.75	**1.00**	**0.98**	**1.00**	**0.94**	**1.00**	0.97	**1.00**	0.90
HPC	0.83	**1.00**	0.71	0.70	**0.87**	0.71	**0.86**	**0.74**	**0.87**	0.71	0.82	0.66
Zookeeper	0.98	0.85	0.92	0.78	**1.00**	0.66	**0.96**	0.80	**1.00**	0.63	**0.96**	**0.82**
HealthApp	0.71	**0.98**	0.90	0.81	**0.99**	0.61	0.97	0.81	**0.99**	0.62	**0.97**	**0.84**
Hadoop	0.73	0.69	0.38	0.33	**0.93**	0.79	**0.96**	**0.72**	0.92	0.82	0.94	**0.72**
Spark	0.45	0.95	0.41	0.34	**1.00**	**1.00**	0.88	**0.74**	**1.00**	**1.00**	**0.89**	0.71
BGL	0.28	0.94	0.43	0.42	0.90	**0.95**	0.89	**0.77**	0.90	**0.95**	**0.91**	**0.77**
Linux	0.20	0.23	0.66	0.40	0.81	0.70	0.87	0.48	0.82	**0.72**	**0.93**	**0.70**
Mac	0.55	0.44	0.48	0.28	0.78	0.53	0.84	**0.54**	**0.87**	**0.63**	**0.85**	**0.54**
Thunderbird	0.51	0.40	0.18	0.10	**0.83**	**0.58**	0.79	0.47	0.79	0.53	**0.84**	**0.54**
Average	0.60	0.74	0.57	0.47	0.88	0.78	0.90	0.74	**0.92**	**0.82**	**0.93**	**0.76**

that demonstrates this effect. All other EM-Merge modules were kept enabled for these runs so that the timing reflects realistic, postprocessing-enabled parsing.

From this experiment, we draw two practical conclusions relevant to RQ2. First, the choice of ϵ has a large impact on end-to-end parse time: in our HPC runs $\epsilon = 0.3$ produced the smallest parse time (164.62 s), substantially faster than both tighter ($\epsilon = 0.2$) and looser ($\epsilon \geq 0.4$) thresholds. Second, core accuracy metrics (GA, PA, FGA, FTA) remain broadly stable across the tested ϵ range, indicating that $\epsilon = 0.3$ offers a favorable trade-off—substantial efficiency gains with no meaningful drop in parsing quality. Intuitively, too-small ϵ can harm downstream stability by missing true candidates and increasing downstream corrective work, while too-large ϵ increases candidate counts and similarity-comparison cost without improving merges.

Table 3: Effect of semantic distance threshold ϵ on HPC

ϵ	parse_time	GA	PA	FGA	FTA
0.2	477.04	0.85	0.691	0.78	0.61
0.3	**164.62**	0.845	0.692	0.777	0.619
0.4	777.32	0.845	0.691	0.777	0.619
0.5	780.47	0.845	0.691	0.777	0.604

In short, integrating EM-Merge improves end-to-end throughput while preserving or improving parsing quality, provided hyperparameters such as ϵ are selected via the described tuning procedure.

5.3 RQ3: Module Contribution

To evaluate the contribution of individual components within the EM-Merge framework, we conducted a series of ablation studies using the HPC dataset. This dataset was chosen due to its high structural and semantic complexity, broad coverage of message types, and strong representativeness of mission-critical computing environments. Its logs combine technical and human-readable messages, display significant lexical variation, use specialized jargon, and include both strictly formatted and free-form content, making it an ideal benchmark for evaluating system components.

In particular, HPC logs are generated by large-scale high-performance computing systems that interleave technical messages (e.g., MPI operations, node failures, GPU management) with human-readable status and error reports. These logs exhibit a high degree of lexical variation, frequent use of domain-specific jargon, and a mixture of rigidly structured and free-form messages. Table 4 summarizes the observed performance changes when semantic embedding, longest common subsequence (LCS) filtering, and the merging stage were selectively removed.

Table 4: Ablation Study Results on HPC Dataset with Relative Changes

Configuration	GA	PA	FGA	FTA
Full EM-Merge	0.871	0.714	0.823	0.658
w/o Semantic	0.845 (-2.99%)	0.646 (-9.53%)	0.780 (-5.22%)	0.596 (-9.42%)
w/o LCS Filtering	0.845 (-2.99%)	0.685 (-4.06%)	0.777 (-5.59%)	0.590 (-10.33%)
w/o Merging	0.845 (-2.99%)	0.692 (-3.08%)	0.783 (-4.86%)	0.623 (-5.32%)

Note: Relative percentages in parentheses indicate performance drops with respect to the full EM-Merge configuration. GA = Grouping Accuracy, PA = Parsing Accuracy, FGA = F1-Grouping Accuracy, FTA = F1-Template Accuracy.

When the semantic embedding module was excluded, the FTA score dropped from 0.658 to 0.596, representing a 9.42% relative decrease. Parsing accuracy (PA) also declined by 9.53%, indicating that semantic information plays a central role in resolving lexical variability and enabling the alignment of functionally equivalent logs, such as `"Node <*> failure"` and `"Compute node <*> crashed"`. The FGA score also dropped by over 5%, confirming that many semantically similar instances failed to cluster properly without embedding-based screening.

Removing the LCS filtering stage led to an even larger decline in FTA, dropping to 0.590 (a 10.33% decrease). This suggests that while semantic embedding enables high-recall candidate generation, structural alignment through LCS is critical to improving precision during merge decisions. The FGA drop of 5.59% further supports the view that LCS filtering reduces structural fragmentation and over-grouping errors.

Disabling the merging stage resulted in the smallest performance degradation, with FTA falling by 5.32% and PA by just 3.08%. Nevertheless, merging remains essential for consolidating structurally redundant templates and compressing the output space for downstream tasks. Its absence still introduced measurable fragmentation, as evidenced by a 4.86% drop in FGA.

Across all metrics, the complete EM-Merge pipeline outperformed any ablated variant. These results reaffirm the complementary roles of semantic embedding, structural refinement, and hierarchical merging. Embedding identifies high-recall candidates; LCS filtering aligns them structurally; and merging consolidates clusters into concise, meaningful templates. Together, they form a robust and generalizable approach for parsing log data in semantically rich and structurally diverse systems such as HPC platforms.

Across diverse datasets and evaluation criteria, EM-Merge demonstrates significant gains in both accuracy and efficiency. Its modular design proves robust to parameter variations, and ablation studies confirm the necessity of each component. These results validate EM-Merge as a practical, high-performance postprocessing enhancement for LLM-based log parsing systems.

6 Discussion

6.1 Efficiency and Practicality

To assess the practical viability and architectural soundness of EM-Merge, we conducted targeted ablation studies on the HPC dataset, a challenging benchmark characterized by its high semantic diversity, structural variability, and operational significance. The results in Table 4 highlight that each module in EM-Merge contributes non-trivially to both parsing accuracy and output coherence.

6.2 Deployment and Practical Implications

Without the need for further LLM inference, model retraining, or supervision, EM-Merge is implemented as a modular postprocessing layer that enhances current log parsers. Its compatibility with both traditional parsing algorithms and LLM-generated outputs allows seamless integration into existing infrastructure. In production settings, EM-Merge has proven effective in consolidating near-duplicate templates, especially in large-scale environments like HPC systems, where template redundancy can hinder performance monitoring and anomaly detection. Moreover, the method's semantic sensitivity and structure-aware design improve both human interpretability and downstream indexing efficiency, providing operational value beyond raw parsing accuracy.

6.3 Limitations and Future Work

Several limitations are worth discussing. First, EM-Merge's evaluation depends on the quality of ground-truth template labels, which may be incomplete or inconsistent in real-world datasets. Second, the semantic screening part, which is based on Sentence-BERT embeddings that have already been trained, may have trouble with extremely domain-specific jargon and may overlook equivalencies that general-purpose language models miss. Third, the current implementation of DBSCAN clustering uses fixed hyperparameters that may not generalize optimally across all log domains. We note that we were unable to test EM-Merge built directly on the very latest large models (for example, GPT-5) due to resource and access constraints; doing so could plausibly reduce fragmentation and improve confidence signals, but those potential gains remain to be empirically validated.

Future work could address these challenges by incorporating adaptive semantic models trained on domain-specific corpora, exploring active learning techniques for robust template labeling, and designing dynamic clustering algorithms that adjust sensitivity based on log structure and entropy. In particular, a natural next step is to evaluate EM-Merge in conjunction with state-of-the-art LLMs, either via few-shot prompting, lightweight fine-tuning, or adapter layers, to study how stronger contextual understanding and calibrated confidences affect consolidation quality. Additionally, extending EM-Merge to support multilingual log streams and integrating it into end-to-end observability pipelines are promising directions for scaling its practical impact.

7 Related Work

Prior research on log parsing can be grouped into three broad families. Early rule-based and heuristic parsers (e.g., Drain) rely on token positions, delimiters and handcrafted regular expressions to generate templates but struggle to adapt to evolving formats and noisy logs [9,13]. Frequency- and pattern-mining approaches (e.g., Logram) detect common substrings or n-grams as candidate templates, while clustering-based methods group similar lines using token-level distances; these methods reduce manual effort but are sensitive to thresholds and often miss semantic equivalence across syntactic variants [14,11]. More recently, semantic and learning-based parsers (for example, UniParser and other neural approaches) use learned representations to distinguish constants from parameters, improving generalization at the cost of requiring labeled data or expensive training and being brittle under domain shift [12,10]. Concurrently, large language models have been applied to template abstraction via zero- and few-shot prompting, demonstrating strong per-instance understanding but introducing new issues such as template fragmentation and high inference cost when used naively [5,19]. EM-Merge differs from these lines of work by acting as a lightweight, model-independent postprocessing layer: it leverages semantic embeddings (Sentence-BERT) and LLM-derived confidence signals to identify and reliably merge semantically equivalent templates without issuing extra LLM

calls, thereby addressing fragmentation and improving cache efficiency while remaining compatible with both traditional and LLM-based parsers [7,17].

8 Conclusion

In this paper, we introduced EM-Merge, a novel embedding-aware and confidence-guided framework designed to enhance the quality and coherence of log templates generated by LLM-based parsers. EM-Merge uses a five-stage process that combines semantic embedding, efficient retrieval, confidence-weighted similarity scoring, clustering, and historical consolidation to systematically address key challenges, including LLM's misinterpretation of punctuation and semantic cues and the fragmentation of semantically similar logs into overlapping templates. Experimental results on 14 Loghub-2.0 datasets demonstrate that EM-Merge consistently outperforms both traditional and state-of-the-art LLM-augmented log parsers. Its modular, LLM-independent architecture enables seamless integration with existing frameworks without extra language model calls or retraining, effectively unifying semantic and structural information to produce more compact, coherent log representations suitable for real-time, high-throughput environments.

Acknowledgments. This work was supported by the National Innovation and Entrepreneurship Training Program for College Students, the Beijing University of Technology Education and Teaching Research Project (No. ER2024KCA01), the open project of the IoT Standards and Application Key Laboratory of the Ministry of Industry and Information Technology (No. 202407), and the Spark Plan Key Project of Beijing University of Technology. The authors would like to express their sincere gratitude to these programs for their financial support. We also thank the faculty and staff at Beijing University of Technology for their guidance and assistance throughout the research.

References

1. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Detecting large-scale system problems by mining console logs. In: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, pp. 117–132 (2009)
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 1285–1298 (2017)
3. Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., et al.: Robust log-based anomaly detection on unstable log data. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 807–817 (2019)
4. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language Models are

Few-Shot Learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901. Curran Associates, Inc. (2020). https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf
5. Zhi, C., Cheng, L., Liu, M., Zhao, X., Xu, Y., Deng, S.: LLM-powered zero-shot online log parsing. In: 2024 IEEE International Conference on Web Services (ICWS), pp. 877–887. IEEE (2024)
6. Xiao, T., Quan, Z., Wang, Z.-J., Zhao, K., Liao, X.: LPV: A Log Parser Based on Vectorization for Offline and Online Log Parsing. In: Proceedings of the 2020 IEEE International Conference on Data Mining (ICDM), pp. 1346–1351 (2020). https://doi.org/10.1109/ICDM50108.2020.00175
7. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992. Association for Computational Linguistics, Hong Kong, China (2019). https://doi.org/10.18653/v1/D19-1410.
8. Jiang, Z., Liu, J., Huang, J., Li, Y., Huo, Y., Gu, J., Chen, Z., Zhu, J., Lyu, M.R.: A large-scale evaluation for log parsing techniques: How far are we? In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 223–234 (2024)
9. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An online log parsing approach with fixed depth tree. In: 2017 IEEE International Conference on Web Services (ICWS), pp. 33–40. IEEE (2017)
10. Ma, J., Liu, Y., Wan, H., Sun, G.: Automatic parsing and utilization of system log features in log analysis: A survey. Applied Sciences **13**(8), 4930 (2023). https://doi.org/10.3390/app13084930
11. Jiang, Z.M., Hassan, A.E., Flora, P., Hamann, G.: Abstracting execution logs to execution events for enterprise applications (short paper). In: 2008 The Eighth International Conference on Quality Software, pp. 181–186. IEEE (2008)
12. Liu, Y., Zhang, X., He, S., Zhang, H., Li, L., Kang, Y., Xu, Y., Ma, M., Lin, Q., Dang, Y., et al.: UniParser: A unified log parser for heterogeneous log data. In: Proceedings of the ACM Web Conference 2022, pp. 1893–1901 (2022)
13. Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z., Lyu, M.R.: Tools and Benchmarks for Automated Log Parsing. In: Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 121–130 (2019). https://doi.org/10.1109/ICSE-SEIP.2019.00021
14. Dai, H., Li, H., Chen, C.-S., Shang, W., Chen, T.-H.: Logram: Efficient log parsing using n-gram dictionaries. IEEE Transactions on Software Engineering **48**(3), 879–892 (2020)
15. Guo, W., Mu, D., Xu, J., Su, P., Wang, G., Xing, X.: LEMNA: Explaining Deep Learning based Security Applications. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18), pp. 364–379. Association for Computing Machinery, Toronto, Canada (2018). https://doi.org/10.1145/3243734.3243792
16. Hu, C., Sun, X., Dai, H., Zhang, H., Liu, H.: Research on Log Anomaly Detection Based on Sentence-BERT. Electronics **12**(17), 3580 (2023) https://doi.org/10.3390/electronics12173580
17. Jiang, Z., Liu, J., Chen, Z., Li, Y., Huang, J., Huo, Y., He, P., Gu, J., Lyu, M.R.: LILAC: Log Parsing using LLMs with Adaptive Parsing Cache. Proceedings of the ACM on Software Engineering **1**(FSE), Article 7, 137–160 (2024). https://doi.org/10.1145/3643733.

18. Wu, X., Li, H., Khomh, F.: On the effectiveness of log representation for log-based anomaly detection. Empirical Software Engineering, 28, 137 (2023). https://doi.org/10.1007/s10664-023-10364-1.
19. Le, V.H., Zhang, H.: Log parsing with prompt-based few-shot learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2438–2449. IEEE (2023)
20. Ma, Z., Chen, A.R., Kim, D.J., Chen, T.P., Wang, S.: LLMParser: An Exploratory Study on Using Large Language Models for Log Parsing. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE), Lisbon, Portugal, pp. 1209–1221 (2024). https://doi.org/10.1145/3597503.3639150
21. Zhong, A., Mo, D., Liu, G., Liu, J., Lu, Q., Zhou, Q., Wu, J., Li, Q., Wen, Q.: LogParser-LLM: Advancing Efficient Log Parsing with Large Language Models. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24), pp. 4559–4570. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3637528.3671810
22. Le, V.-H., Zhang, H.: Log Parsing: How Far Can ChatGPT Go? In: Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1699–1704 (2023). https://doi.org/10.1109/ASE56229.2023.00206
23. Zhou, Y., Chen, Y., Rao, X., Zhou, Y., Li, Y., Hu, C.: Leveraging Large Language Models and BERT for Log Parsing and Anomaly Detection. Mathematics **12**, 2758 (2024). https://doi.org/10.3390/math12172758
24. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. In: IEEE Transactions on Big Data **7**(3), 535–547 (2019)
25. Son, M.: Density-based Algorithms for Active and Anytime Clustering. Master's thesis, Ludwig-Maximilians-Universität München, Munich (2014). https://edoc.ub.uni-muenchen.de/17533/1/Mai_Son_T.pdf
26. Khan, Z.A., Shin, D., Bianculli, D., Briand, L.: Guidelines for Assessing the Accuracy of Log Message Template Identification Techniques. In: Proceedings of the 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), pp. 1095–1106 (2022). https://doi.org/10.1145/3510003.3510101

This version of the contribution has been accepted for publication, after peer review (when applicable) but is not the Version of Record and does not reflect post-acceptance improvements, or any corrections.
The Version of Record is available online at: https://doi.org/10.1007/978-981-95-7826-9_17.
Use of this Accepted Version is subject to the publisher's Accepted Manuscript terms of use: https://www.springernature.com/gp/open-research/policies/accepted-manuscript-terms